

Вопросы к экзамену

1. Алгоритм Форда-Фалкерсона нахождения максимального потока в графе

Алгоритм Форда — Фалкерсона решает задачу нахождения максимального потока в транспортной сети. , вдоль которого можно послать большой поток). Процесс повторяется, пока можно найти увеличивающий путь.

Как работает сам алгоритм?

- Следует понимать остаточную сеть как тот же граф, который имеется на входе, но в этом случае мы будем производить над ним некоторые операции:
- Отправлять определенное количество потока из текущей вершины в соседние.
- Возвращать определенное количество потока из соседних вершин в текущую.
- В начальный момент времени поток, который мы хотим провести через нашу сеть, должен быть равен нулю. Остаточная сеть совпадает с исходной сетью.
- Находим любой путь из истока в сток в остаточной сети. Если путь не находим, утверждается, что поток является максимальным.
- Пускаем через найденный путь поток равный минимальному весу ребра, которое входит в множество рёбер найденного пути.
- Из веса рёбер на этом пути высчитываем размер потока, который мы пустили.
- А к весу обратных рёбер (будем считать, что они существуют в остаточной сети и равны 0) прибавляем размер потока. Другими словами, на предыдущем шаге мы отправили некоторое количество потока из текущей вершины в следующую, а теперь при желании можем вернуть это же количество потока обратно в текущую.
- Возвращаемся обратно к нахождению пути в остаточной сети после модификации.

2. Алгоритм поиска в ширину (BFS- Breadth-First Search)

1. Поместить узел, с которого начинается поиск, в изначально пустую очередь.
2. Извлечь из начала очереди узел **u** и пометить его как развёрнутый.
 - a. Если узел **u** является целевым узлом, то завершить поиск с результатом «успех».
 - b. В противном случае, в конец очереди добавляются все преемники узла **u**, которые ещё не развёрнуты и не находятся в очереди.
3. Если очередь пуста, то все узлы связного графа были просмотрены, следовательно, целевой узел недостижим из начального; завершить поиск с результатом «неудача».
4. Вернуться к п. 2.

3. Алгоритм поиска в глубину (DFS)

- Поиск в глубину (Deep First Search, DFS) — алгоритм обхода графа, который движется от начальной вершины как можно дальше сначала по первому ребру, потом по второму, и так далее Поиск в глубину можно использовать для перебора вершин, чтобы найти вершину с заданным значением.

• Алгоритм поиска в глубину

Пусть задан граф $G=(V,E)$, где **V** — множество вершин графа, **E** — множество ребер графа. Предположим, что в начальный момент времени все вершины графа окрашены в *белый* цвет. Выполним следующие действия:

1. Пройдём по всем вершинам $v \in V$
 - Если вершина **V** белая, выполним для неё DFS(**v**).

Процедура DFS (параметр — вершина $u \in V$)

1. Перекрашиваем вершину **u** в *серый* цвет.
2. Для всякой вершины **w**, смежной с вершиной **u** и окрашенной в белый цвет, рекурсивно выполняем процедуру DFS(**w**)

3. Перекрашиваем вершину **u** в **чёрный** цвет.

4. Высказывания.

Высказывания в логике — это предложения, которые могут быть либо истинными, либо ложными. В математической логике и дискретной математике высказывания играют ключевую роль, так как они являются основой для построения логических выражений, формул и доказательств.

Определение высказывания

Высказывание — это предложение, которое имеет определенное значение истинности, т.е. оно может быть либо истинным (Т), либо ложным (F). Важно отметить, что высказывание не может быть одновременно истинным и ложным.

Примеры высказываний

1. Простые высказывания:

- " $2 + 2 = 4$ " — это истинное высказывание.
- " $3 > 5$ " — это ложное высказывание.

2. Составные высказывания:

- "Если идет дождь, то улицы мокрые" — это составное высказывание, состоящее из двух простых высказываний: "Идет дождь" и "Улицы мокрые".

Логические операторы

Для построения составных высказываний используются логические операторы:

1. **Отрицание ($\neg$):** Инвертирует значение истинности высказывания.
 - Пример: Если p — "Идет дождь" (истинно), то $\neg p$ — "Не идет дождь" (ложно).
 2. **Конъюнкция ($\wedge$):** Истинно, если оба высказывания истинны.
 - Пример: $p \wedge q$ — "Идет дождь и улицы мокрые".
 3. **Дизъюнкция ($\vee$):** Истинно, если хотя бы одно из высказываний истинно.
 - Пример: $p \vee q$ — "Идет дождь или светит солнце".
 4. **Импликация ($\rightarrow$):** Истинно, если из истинности первого высказывания следует истинность второго.
 - Пример: $p \rightarrow q$ — "Если идет дождь, то улицы мокрые".
 5. **Эквиваленция ($\leftrightarrow$):** Истинно, если оба высказывания имеют одинаковую истинность.
 - Пример: $p \leftrightarrow q$ — "Идет дождь тогда и только тогда, когда улицы мокрые"
-

5. Формулы логики высказываний

Основные элементы формул логики высказываний

1. Атомарные высказывания:

- Обозначаются символами p, q, r и т.д.
- Представляют простые утверждения, которые могут быть истинными или ложными.

2. Логические операторы:

- **Отрицание ($\neg$):** Инвертирует значение высказывания.
- **Конъюнкция ($\wedge$):** Логическое "И". Истинно, если оба высказывания истинны.
- **Дизъюнкция ($\vee$):** Логическое "ИЛИ". Истинно, если хотя бы одно из высказываний истинно.
- **Импликация ($\rightarrow$):** Логическое "ЕСЛИ...ТО". Истинно, если из истинности первого высказывания следует истинность второго.
- **Эквиваленция ($\leftrightarrow$):** Логическое "ЕСЛИ И ТОЛЬКО ЕСЛИ". Истинно, если оба высказывания имеют одинаковую истинность.

6. Формализованное представление рассуждений в логике высказываний.

Формализованное представление рассуждений в логике высказываний

Формализованное представление рассуждений в логике высказываний заключается в использовании логических формул и правил вывода для построения доказательств и проверки логической правильности рассуждений. Это позволяет избегать двусмысленности и неточностей, характерных для естественного языка, и обеспечивает строгую проверку логической корректности.

Основные элементы формализованных рассуждений

1. **Атомарные высказывания:** Обозначаются символами p, q, r и т.д. Представляют простые утверждения, которые могут быть истинными или ложными.
2. **Логические операторы:**
 - Отрицание ($\neg$)
 - Конъюнкция ($\wedge$)
 - Дизъюнкция ($\vee$)
 - Импликация ($\rightarrow$)
 - Эквиваленция ($\leftrightarrow$)
3. **Логические формулы:** Синтаксически корректные выражения, составленные из атомарных высказываний и логических операторов.
4. **Правила вывода:** Логические правила, по которым из одних формул можно получать другие. Примеры правил вывода включают:
 - Модус поненс (Modus Ponens)
 - Модус толленс (Modus Tollens)
 - Правило цепи (Hypothetical Syllogism)
 - Правило исключенного третьего (Law of Excluded Middle)

Примеры правил вывода

1. **Модус поненс (Modus Ponens):**
 - **Схема:**
 1. $p \rightarrow q$
 2. p
 3. Следовательно, q
 - **Пример:**
 1. Если идет дождь, то улицы мокрые.
 2. Идет дождь.
 3. Следовательно, улицы мокрые.
2. **Модус толленс (Modus Tollens):**
 - **Схема:**
 1. $p \rightarrow q$
 2. $\neg q$
 3. Следовательно, $\neg p$
 - **Пример:**
 1. Если идет дождь, то улицы мокрые.
 2. Улицы не мокрые.
 3. Следовательно, не идет дождь.
3. **Правило цепи (Hypothetical Syllogism):**
 - **Схема:**
 1. $p \rightarrow q$
 2. $q \rightarrow r$
 3. Следовательно, $p \rightarrow r$
 - **Пример:**
 1. Если идет дождь, то улицы мокрые.
 2. Если улицы мокрые, то люди открывают зонтики.
 3. Следовательно, если идет дождь, то люди открывают зонтики.

Структура формализованного рассуждения

1. **Посылки:** Исходные утверждения, которые считаются истинными.
2. **Правила вывода:** Логические правила, используемые для получения новых утверждений из посылок.
3. **Заключение:** Утверждение, которое выводится из посылок с помощью правил вывода.

Пример формализованного рассуждения

Рассмотрим пример формализованного рассуждения, использующего правила логики.

Задача: Доказать, что если $p \rightarrow q$ и $q \rightarrow r$, то $p \rightarrow r$.

1. **Посылки:**

- $p \rightarrow q$
- $q \rightarrow r$

2. Применение правила вывода:

- Согласно правилу цепи (Hypothetical Syllogism), если $p \rightarrow q$ и $q \rightarrow r$, $p \rightarrow r$.

3. Заключение:

- $p \rightarrow r$

Таким образом, мы формализованно доказали утверждение.

Формализованное представление в логике высказываний

Формализованное представление рассуждений в логике высказываний позволяет четко и однозначно строить доказательства, использовать логические правила для вывода новых утверждений и проверять логическую правильность рассуждений. Это важно в математике, информатике, философии и других областях, где требуется строгий анализ логических связей и построение корректных аргументов.

7. Проверка рассуждений на логическую правильность.

Проверка рассуждений на логическую правильность заключается в том, чтобы убедиться, что заключение логически следует из посылок. Это можно сделать с помощью нескольких методов, включая истинностные таблицы, использование логических эквивалентов и законов, а также построение формализованных доказательств с использованием правил вывода.

Основные методы проверки логической правильности

1. Истинностные таблицы:

- Метод заключается в построении таблицы, в которой вычисляются значения истинности всех посылок и заключения для всех возможных значений атомарных высказываний. Если для всех случаев, когда посылки истинны, заключение также истинно, то рассуждение логически правильно.

2. Правила вывода:

- Применение правил вывода для получения заключения из посылок. Если заключение можно вывести из посылок с использованием правил логики, то рассуждение логически правильно.

3. Логические эквиваленты:

- Преобразование посылок и заключения с использованием логических эквивалентов и законов логики (таких как законы де Моргана, дистрибутивные законы и т.д.) для упрощения проверки логической связи между ними.
-

8. Основные законы логики высказываний.

В логике высказываний существуют несколько фундаментальных законов, которые определяют, как логические операции взаимодействуют друг с другом. Эти законы играют важную роль в построении и анализе логических выражений и рассуждений.

1. Закон двойного отрицания

- Формула: $\neg(\neg p) \leftrightarrow p$
- Описание: Отрицание отрицания высказывания эквивалентно самому высказыванию.
- Пример: "Не (не идет дождь)" эквивалентно "Идет дождь".

2. Законы идемпотентности

- Формулы:
 - $p \wedge p \leftrightarrow p$
 - $p \vee p \leftrightarrow p$
- Описание: Конъюнкция или дизъюнкция высказывания с самим собой эквивалентна самому высказыванию.
- Пример: "Идет дождь и идет дождь" эквивалентно "Идет дождь".

3. Законы коммутативности

- Формулы:
 - $p \wedge q \leftrightarrow q \wedge p$
 - $p \vee q \leftrightarrow q \vee p$

- Описание: Порядок высказываний в конъюнкции или дизъюнкции не влияет на их истинность.
- Пример: "Идет дождь и светит солнце" эквивалентно "Светит солнце и идет дождь".

4. Законы ассоциативности

- Формулы:
 - $(p \wedge q) \wedge r \leftrightarrow p \wedge (q \wedge r)$
 - $(p \vee q) \vee r \leftrightarrow p \vee (q \vee r)$
- Описание: Группировка высказываний в конъюнкции или дизъюнкции не влияет на их истинность.
- Пример: "(Идет дождь и светит солнце) и идет снег" эквивалентно "Идет дождь и (светит солнце и идет снег)".

5. Законы дистрибутивности

- Формулы:
 - $p \wedge (q \vee r) \leftrightarrow (p \wedge q) \vee (p \wedge r)$
 - $p \vee (q \wedge r) \leftrightarrow (p \vee q) \wedge (p \vee r)$
- Описание: Конъюнкция и дизъюнкция могут распределяться друг относительно друга.
- Пример: "Идет дождь и (светит солнце или идет снег)" эквивалентно "(Идет дождь и светит солнце) или (Идет дождь и идет снег)".

6. Законы де Моргана

- Формулы:
 - $\neg(p \wedge q) \leftrightarrow \neg p \vee \neg q$
 - $\neg(p \vee q) \leftrightarrow \neg p \wedge \neg q$
- Описание: Отрицание конъюнкции эквивалентно дизъюнкции отрицаний, и отрицание дизъюнкции эквивалентно конъюнкции отрицаний.
- Пример: "Не (идет дождь и светит солнце)" эквивалентно "Не идет дождь или не светит солнце".

7. Закон исключенного третьего

- Формула: $p \vee \neg p$
- Описание: Любое высказывание либо истинно, либо ложно.
- Пример: "Идет дождь или не идет дождь" всегда истинно.

8. Закон непротиворечия

- Формула: $\neg(p \wedge \neg p)$
- Описание: Высказывание не может быть одновременно истинным и ложным.
- Пример: "Не (идет дождь и не идет дождь)" всегда истинно.

9. Понятие булевых функций.

Булева функция – это функция, и аргументы и значение которой принадлежит множеству $\{0, 1\}$.

Булева функция — это функция, которая принимает значения на входе из множества $\{0, 1\}$ и возвращает значения из этого же множества. Булевы функции используются в различных областях, включая математику, информатику, цифровую электронику и логику.

Основные понятия булевых функций

1. **Определение:** Булева функция f от n переменных — это отображение $f: \{0, 1\}^n \rightarrow \{0, 1\}$.
2. **Переменные:** Булева функция может принимать несколько переменных, например $x_1, x_2, \dots, x_n$
3. **Истинностная таблица:** Булева функция может быть описана с помощью истинностной таблицы, которая показывает, как значения функции зависят от всех возможных комбинаций входных значений.

Основные свойства булевых функций

1. **Замкнутость:**

- Булевы функции замкнуты относительно операций конъюнкции, дизъюнкции и отрицания, то есть любые комбинации булевых функций также являются булевыми функциями.

2. Полнота:

- Набор булевых функций называется полным, если с его помощью можно выразить любую булеву функцию. Например, набор {AND, OR, NOT} является полным.

3. Самодвойственность:

- Булева функция f является самодвойственной, если $f(x_1, x_2, \dots, x_n) = \neg f(\neg x_1, \neg x_2, \dots, \neg x_n)$

10. Основные булевы функции от двух переменных.

1. Конъюнкция (AND)
2. Дизъюнкция (OR)
3. Исключающее ИЛИ (XOR), Обозначение: $x \oplus y$
4. Импликация (IMPLIES), Обозначение: $x \rightarrow y$
5. Эквиваленция (EQUIVALENT), Обозначение: $x \leftrightarrow y$
6. Отрицание (NOT)
7. Конъюнкция с отрицанием (NAND), Обозначение: $x \uparrow y$ или $\neg(x \wedge y)$
8. Дизъюнкция с отрицанием (NOR), Обозначение: $x \downarrow y$ или $\neg(x \vee y)$
9. Исключающее НЕ-ИЛИ (XNOR), Обозначение: $\neg(x \oplus y)$

11. СДНФ и СКНФ. ДНФ, минимальные ДНФ.

СДНФ — это форма представления булевой функции, в которой функция выражена как дизъюнкция (логическое "или") конъюнктов (логическое "и") всех возможных комбинаций переменных, при которых функция принимает значение истина (1).

1. **Определение:** Булева функция в СДНФ представляется как дизъюнкция минтермов, где каждый минтерм — это конъюнкция всех переменных или их отрицаний.

СКНФ — это форма представления булевой функции, в которой функция выражена как конъюнкция (логическое "и") дизъюнктов (логическое "или") всех возможных комбинаций переменных, при которых функция принимает значение ложь (0).

1. **Определение:** Булева функция в СКНФ представляется как конъюнкция макстермов, где каждый макстерм — это дизъюнкция всех переменных или их отрицаний.

ДНФ — это форма представления булевой функции, в которой функция выражена как дизъюнкция конъюнктов. В отличие от СДНФ, в ДНФ не обязательно использовать все возможные комбинации переменных.

1. **Определение:** Булева функция в ДНФ представляется как дизъюнкция одного или нескольких конъюнктов.

Минимальная ДНФ — это такая форма представления булевой функции, в которой функция выражена как дизъюнкция конъюнктов, но с минимальным числом этих конъюнктов. Цель — упрощение функции без изменения ее логического поведения.

Определение: Булева функция в минимальной ДНФ представляется как дизъюнкция минимально возможного числа конъюнктов.

Пример: Рассмотрим функцию $f(x, y, z)$, которая истинна для значений $(x, y, z) = (1, 0, 0)$, $(1, 0, 1)$, $(0, 1, 1)$ и $(1, 1, 1)$

Обычная ДНФ:

$$f(x, y, z) = (x \wedge \neg y \wedge \neg z) \vee (x \wedge \neg y \wedge z) \vee (\neg x \wedge y \wedge z) \vee (x \wedge y \wedge z)$$

Минимальная ДНФ (упрощенная):

$$f(x, y, z) = x \wedge \neg y \vee y \wedge z$$

12. Понятие формулы над системой булевых функций.

Done!!!!!!!!!!!!!!.

13. Понятие замыкания системы булевых функций, основные замкнутые классы булевых функций.

Определение

Замыкание системы булевых функций: Пусть F – некоторое множество булевых функций. Замыкание этого множества, обозначаемое как $\overline{F}$, есть наименьшее множество булевых функций, содержащее F и замкнутое относительно суперпозиции.

Пример

Пусть $F = \{\wedge, \vee, \neg\}$. Замыкание этого множества будет включать все возможные булевы функции, которые можно получить, применяя операции $\wedge$, $\vee$, и $\neg$ в различных комбинациях. Это замыкание будет включать все булевы функции, так как из $\wedge$, $\vee$ и $\neg$ можно составить любую булеву функцию.

Основные замкнутые классы булевых функций

Существуют несколько важных замкнутых классов булевых функций, каждый из которых обладает особыми свойствами. Вот некоторые из них:

1. **Замкнутый класс T_0** (Класс констант):
 - Содержит только константные функции $f(x_1, x_2, \dots, x_n) = 0$ и $f(x_1, x_2, \dots, x_n) = 1$.
2. **Замкнутый класс T_1** (Класс сохранения нуля):
 - Содержит функции, которые принимают значение 0 при хотя бы одном нулевом аргументе.
3. **Замкнутый класс T** (Класс сохранения единицы):
 - Содержит функции, которые принимают значение 1 при хотя бы одном единичном аргументе.
4. **Класс самодвойственных функций (S):**
 - Содержит функции, для которых $f(\neg x_1, \neg x_2, \dots, \neg x_n) = \neg f(x_1, x_2, \dots, x_n)$.
5. **Класс линейных функций (L):**
 - Содержит функции, которые могут быть выражены в виде линейных комбинаций переменных с использованием операции XOR ($\oplus$).
6. **Класс монотонных функций (M):**

Содержит функции, которые сохраняют порядок: если $x_i \leq y_i$ для всех i , то $f(x_1, x_2, \dots, x_n) \leq f(y_1, y_2, \dots, y_n)$

14. Полнота системы булевых функций.

Полнота системы булевых функций означает, что любую булеву функцию можно выразить в терминах данной системы функций. Если система функций полная, то она замкнута относительно суперпозиции (композиции функций).

Для того чтобы система булевых функций была полной, она должна содержать или быть способной выразить следующие элементы:

1. Константы 0 и 1 (или хотя бы одну из них).
2. Операции конъюнкции ($\wedge$) и дизъюнкции ($\vee$).
3. Отрицание ($\neg$).
4. Либо операция XOR (исключающее ИЛИ, $\oplus$), либо операции Штрих Шеффера (NAND) или Стрелка Пирса (NOR).

Проверка полноты

Существует теорема Поста, которая формализует критерии полноты системы булевых функций. Система булевых функций полна, если она не замкнута ни под одним из следующих свойств:

1. Сохранение нуля (константная функция 0).
2. Сохранение единицы (константная функция 1).
3. Самодвойственность (функция $f(\neg x) = \neg f(x)$).
4. Линейность (функции могут быть выражены только в виде суммы модуль 2 (XOR)).
5. Монотонность (если $x \leq y$, то $f(x) \leq f(y)$).

Если система булевых функций не замкнута под каким-либо из этих свойств, она является полной.

15. Теорема Поста.

- Теорема Поста, названная в честь американского математика Эмиля Поста, является важным результатом в теории функций алгебры логики. Теорема Поста описывает все возможные замкнутые классы логических функций, которые можно определить, используя конечное множество базисных логических операций. В контексте дискретной математики и логики, эта теорема важна для понимания, какие комбинации логических операций могут выражать любые возможные логические функции.
- **Формулировка теоремы Поста**
- Теорема Поста утверждает, что если F — это замкнутый класс булевых функций, то F является одним из следующих пяти типов замкнутых классов, или их пересечением:
 1. **Тривиальный класс:**
 - Содержит только константы 0 и 1.
 2. **Класс сохраняющих 0:**
 - Все функции $f \in F$ сохраняют 0, то есть $f(0, \dots, 0) = 0$.
 3. **Класс сохраняющих 1:**
 - Все функции $f \in F$ сохраняют 1, то есть $f(1, \dots, 1) = 1$.
 4. **Класс самодвойственных функций:**
 - Все функции $f \in F$ удовлетворяют условию самодвойственности, то есть $f(x_1, \dots, x_n) = \neg f(\neg x_1, \dots, \neg x_n)$.
 5. **Класс монотонных функций:**
 - Все функции $f \in F$ являются монотонными, то есть если $x \leq y$, то $f(x) \leq f(y)$.
 6. **Класс линейных функций:**
 - Все функции $f \in F$ являются линейными, то есть могут быть выражены с помощью операций XOR и констант.

Пример использования теоремы Поста

Для иллюстрации, давайте рассмотрим, как теорема Поста может применяться для анализа набора логических операций:

Предположим, у нас есть набор базисных операций: $\{\wedge, \vee, \neg\}$. Мы хотим определить, является ли этот набор замкнутым и какие функции можно выразить с его помощью.

- **Проверка замкнутости:**
 - С помощью $\wedge$ и $\vee$ мы можем строить монотонные функции.
 - Операция $\neg$ позволяет выражать не монотонные функции.
 - Мы можем выразить константы 0 и 1 с помощью этих операций.

Таким образом, набор $\{\wedge, \vee, \neg\}$ замкнут относительно всех булевых функций, что означает, что мы можем выразить любую булеву функцию, используя эти операции. Это соответствует тому факту, что набор $\{\wedge, \vee, \neg\}$ является функционально полным.

16. Предикаты.

Предикат — это логическая функция, которая принимает один или несколько аргументов и возвращает логическое значение (истина или ложь). Предикат может быть представлен как $P(x_1, x_2, \dots, x_n)$, где $x_1, x_2, \dots, x_n$ — переменные

Примеры предикатов

1. Одноместный предикат:
 - - $P(x)$: "x является простым числом."
 - - $P(2)$ истинно, так как 2 — простое число.
 - - $P(4)$ ложно, так как 4 — не простое число.
2. Двуместный предикат:
 - - $R(x, y)$: "x больше y."
 - - $R(5, 3)$ истинно, так как 5 больше 3.
 - - $R(2, 7)$ ложно, так как 2 не больше 7.

Кванторы

Для работы с предикатами используются кванторы, которые позволяют выражать общие утверждения о множествах объектов.

1. Универсальный квантор ($\forall$):

- Обозначает, что предикат истинен для всех элементов множества.

- Пример: $(\forall)x P(x)$ означает "Для всех x , $P(x)$ истинно."

2. Существовательный квантор ($\exists$):

- Обозначает, что существует по крайней мере один элемент множества, для которого предикат истинен.

- Пример: $(\exists x P(x))$ означает "Существует x , такой что $P(x)$ истинно."

Примеры с кванторами

1. Универсальный квантор:

- $(\forall)x (x > 0)$: "Все x больше 0."

- Пример: $(\forall) x (x^2 \geq 0)$ истинно, так как квадрат любого числа неотрицателен.

2. Существовательный квантор:

- $(\exists x (x < 0))$: "Существует x , которое меньше 0."

- Пример: $(\exists x (x^2 = 4))$ истинно, так как существует два числа, удовлетворяющих этому условию: $(x = 2)$ и $(x = -2)$.

Сложные выражения с предикатами

Комбинируя предикаты и кванторы, можно строить сложные логические выражения.

$\forall x \exists y (X < Y)$ "Для любого x существует y , такое что x меньше y ."

- $\exists x \forall y (X \neq Y)$: "Существует x , такой что для любого y , x не равно y ."

Свойства предикатов

- Рефлексивность: $\forall x R(x, x)$ для отношения R — отношение рефлексивно, если оно истинно для любого x .

- Симметричность: $\forall x \forall y (R(x, y) \rightarrow R(y, x))$ — отношение симметрично, если $R(x, y)$ влечет $R(y, x)$

- Транзитивность: $\forall x \forall y \forall z ((R(x, y) \wedge R(y, z)) \rightarrow R(x, z))$ — отношение транзитивно, если из $R(x, y)$ и $R(y, z)$ следует $R(x, z)$

17. Операции логики предикатов.

Пусть на некотором множестве M определены два предиката $P(x)$ и $Q(x)$.

Определение. Конъюнкцией двух предикатов $P(x)$ и $Q(x)$ называется предикат $P(x) \wedge Q(x)$, который принимает значение «истина» при тех и только тех значениях $x \in M$, при которых каждый из предикатов принимает значение «истина» и принимает значение «ложь» во всех остальных случаях. Областью истинности предиката $P(x) \wedge Q(x)$ является пересечение областей истинности обоих предикатов, то есть $IP \wedge Q = IP \cap IQ$.

Пример 13. $P(x)$ – « x – четное число», $Q(x)$ – « x кратно 3». Предикат $P(x) \wedge Q(x)$: « x – четное число и x кратно 3» = « x делится на 6».

Определение. Дизъюнкцией двух предикатов $P(x)$ и $Q(x)$ называется предикат $P(x) \vee Q(x)$ который принимает значение «ложь» при тех и только тех значениях, при которых каждый из предикатов принимает значение «ложь» и принимает значение «истина» во всех остальных случаях. Областью истинности предиката $P(x) \vee Q(x)$ является объединение областей истинности обоих предикатов, то есть $IP \vee Q = IP \cup IQ$.

Определение. Отрицанием предиката $P(x)$ называется предикат $P(x)^{\neg}$, который принимает значение «истина» при всех значениях $x \in M$, при которых предикат $P(x)$ принимает значение «ложь», и принимает значение «ложь» при тех значениях $x \in M$, при которых предикат $P(x)$ принимает значение «истина». Областью истинности предиката $P(x)^{\neg}$ является дополнение множества истинности предиката $P(x)$ до множества M , то есть $I\bar{P} = M \setminus IP = CIP$.

Определение. Импликацией предикатов $P(x)$ и $Q(x)$ называется предикат $P(x) \rightarrow Q(x)$, который является ложным при тех и только тех значениях $x \in M$, при которых одновременно $P(x)$ принимает значение «истина», а $Q(x)$ принимает значение «ложь» и принимает значение «истина» во всех остальных случаях. Так как при каждом фиксированном x

справедлива 9 равносильность $P(x) \rightarrow Q(x) \equiv P(x) \vee \neg Q(x)$, то область истинности предиката $P(x) \rightarrow Q(x)$, является объединением дополнения области истинности предиката $P(x)$ до множества M и области истинности предиката $Q(x)$, то есть $IP \rightarrow Q = I\bar{P} \cup IQ$.

Определение. Эквиваленцией предикатов $P(x)$ и $Q(x)$ называется предикат $P(x) \leftrightarrow Q(x)$, который превращается в истинное высказывание при всех $x \in M$, при которых одновременно $P(x)$ и $Q(x)$ принимают одинаковые значения истинности. Так как при каждом фиксированном x справедлива равносильность $P(x) \leftrightarrow Q(x) \equiv (\bar{P} \vee Q) \wedge (P \vee \bar{Q})$ то область истинности предиката $P(x) \leftrightarrow Q(x)$, является конъюнкцией объединений дополнения области истинности предиката $P(x)$ до множества M и области истинности предиката $Q(x)$, и дополнения области истинности предиката $Q(x)$ до множества M и области истинности предиката $P(x)$ то есть $IP \leftrightarrow Q = (I\bar{P} \cup IQ) \cap (I\bar{Q} \cup IP)$

18. Кванторы общности и кванторы существования.

Квантор общности ($\forall$)

Квантор общности ($\forall$) используется для выражения утверждения, которое справедливо для всех элементов множества. Формула с квантором общности имеет вид:

$\forall x P(x)$

где:

- x - переменная,
- $P(x)$ - выражение, зависящее от x .

Примеры:

- $\forall x (x > 0)$ - "для всех x , x больше нуля".
- $\forall n (n^2 \geq 0)$ - "для всех целых чисел n , n^2 больше или равно нулю".

Квантор существования ($\exists$)

Квантор существования ($\exists$) используется для выражения утверждения о существовании хотя бы одного элемента множества, для которого выполняется условие. Формула с квантором существования имеет вид:

$\exists x P(x)$

где:

- x - переменная,
- $P(x)$ - выражение, зависящее от x .

Примеры:

- $\exists x (x > 0)$ - "существует x , такое что x больше нуля".
- $\exists n (n^2 = 4)$ - "существует целое число n , такое что $n = n^2$ " (решение: $n = 2$ или $n = -2$).

Связь и использование кванторов

Кванторы часто используются вместе с предикатами для формулирования более сложных утверждений. Например:

- $\forall x (P(x) \rightarrow Q(x))$ - "для всех x , если $P(x)$, то $Q(x)$ ".
 - $\exists x (P(x) \wedge Q(x))$ - "существует x , такое что $P(x)$ и $Q(x)$ ".
-

19. Формализованное представление рассуждений на языке предикатов.

Формализованное представление рассуждений на языке предикатов предполагает использование логических выражений, включающих предикаты, кванторы и переменные. Давайте рассмотрим основные элементы и примеры формализованных рассуждений на языке предикатов.

Основные элементы

1. **Переменные:** Обозначения, которые представляют объекты или элементы множества, с которыми работает предикат.
2. **Предикаты:** Утверждения, которые зависят от переменных и имеют логическое значение истинности или ложности. Примеры предикатов могут быть такими:
 - $P(x)$ - " x является чётным числом".
 - $Q(x, y)$ - " x больше y ".
 - $R(x, y, z)$ - " x , y и z образуют треугольник".

3. Кванторы: Уточняют область применения переменных в выражении.

- **Квантор общности ($\forall$):** "Для всех" или "для каждого". Используется для утверждений, которые справедливы для всех элементов множества.
- **Квантор существования ($\exists$):** "Существует". Используется для утверждений о существовании хотя бы одного элемента, удовлетворяющего условию.

Примеры формализованных рассуждений

1. Пример с квантором общности:

- Утверждение: "Все люди существуют".
- Формализация: $\forall x \exists y (\text{Человек}(x) \rightarrow \text{Существует}(y))$
 - Здесь $\text{Человек}(x)$ - предикат, означающий, что x является человеком.
 - $\text{Существует}(y)$ - предикат, означающий, что y существует.

2. Пример с квантором существования:

- Утверждение: "Существует человек, который любит математику".
- Формализация: $\exists x (\text{Человек}(x) \wedge \text{Любит}(x, \text{Математика}))$
 - Здесь $\text{Любит}(x, \text{Математика})$ - предикат, означающий, что x любит математику.

3. Сложный пример с обоими кванторами:

- Утверждение: "Есть человек, который любит всех животных".
- Формализация: $\exists x (\text{Человек}(x) \wedge \forall y (\text{Животное}(y) \rightarrow \text{Любит}(x, y)))$
 - Здесь $\text{Животное}(y)$ - предикат, означающий, что y является животным.

20. Запись математических утверждений на языке предикатов.

На языке предикатов можно записывать различные математические утверждения и чисто формально используя логику предикатов получать отрицания к этим утверждениям.

Запись математических утверждений на языке предикатов позволяет формализовать их с использованием переменных, предикатов и кванторов. Давайте рассмотрим основные элементы и примеры записи математических утверждений на языке предикатов.

Основные элементы. (As written in the 19)

21. Основные подходы к определению алгоритмов.

Определение алгоритмов является фундаментальной задачей теории вычислений и информатики. Существует несколько основных подходов к определению алгоритмов, каждый из которых предоставляет формальную структуру для анализа и описания алгоритмических процессов. Вот основные подходы:

1. Машина Тьюринга

Машина Тьюринга – это абстрактная математическая модель, предложенная Аланом Тьюрингом в 1936 году. Она состоит из бесконечной ленты, разделённой на ячейки, в каждой из которых может быть записан символ из конечного алфавита, головки для чтения и записи, а также таблицы переходов, определяющей действия машины в зависимости от текущего состояния и символа под головкой.

Основные компоненты машины Тьюринга:

- **Лента:** потенциально бесконечная последовательность ячеек.
- **Головка:** устройство для чтения и записи символов на ленте.
- **Алфавит:** конечное множество символов, используемых на ленте.
- **Состояния:** конечное множество состояний машины.
- **Таблица переходов:** функция, определяющая следующее состояние и действие машины (запись символа, движение ленты) в зависимости от текущего состояния и символа под головкой.

2. Лямбда-исчисление

Лямбда-исчисление – формальная система, введенная Алонзо Чёрчем в 1930-х годах, используется для исследования функций, их применения и рекурсии. Это математическая абстракция вычисления с использованием функций как первичных объектов.

Основные понятия лямбда-исчисления:

- **Переменные:** символы, представляющие значения.

- **Абстракция:** выражения вида $\lambda x.E$, где x – переменная, а E – выражение.
- **Применение:** выражения вида $(E_1 E_2)$, где E_1 и E_2 – выражения.

3. Рекурсивные функции

Рекурсивные функции – это класс функций, которые определяются с использованием базовых функций и операций рекурсии. Они были изучены Клини и Гёделем.

Основные типы рекурсивных функций:

- **Примитивно-рекурсивные функции:** функции, определённые с использованием базовых функций (нулевая функция, функция следующего, проекции) и операций примитивной рекурсии.
- **Общие рекурсивные функции:** более общий класс, включающий примитивно-рекурсивные функции и операции минимизации.

22. Частично-рекурсивные функции.

С помощью оператора минимизации из примитивно рекурсивных, можем получать функции, которые не везде определены, они называются частично-рекурсивными

Класс частично рекурсивных функций шире, чем класс примитивно-рекурсивных. Тезис Черча заключается в том, что любая вычислимая функция является частично рекурсивной

Частично-рекурсивные функции – это класс функций, определяемых в теории вычислимости и теории рекурсивных функций. В отличие от примитивно-рекурсивных функций, частично-рекурсивные функции могут быть определены не на всех входных значениях, то есть они могут быть не везде определены (частичны)

Свойства частично-рекурсивных функций: 1. Частичность, 2. Выразительная мощность, 3. Связь с машинами Тьюринга.

23. Машина Тьюринга.

Машина Тьюринга – это абстрактная математическая модель вычислительной машины, предложенная английским математиком Аланом Тьюрингом в 1936 году. Она служит одной из основ теории вычислимости и используется для формального определения понятий алгоритма и вычислимой функции. Машина Тьюринга состоит из бесконечной ленты, головы для чтения/записи и управляющего устройства.

Основные компоненты машины Тьюринга

1. **Лента:**
 - Бесконечная в обе стороны.
 - Разделена на ячейки, каждая из которых может содержать символ из конечного алфавита.
 - Алфавит обычно включает пустой символ (обычно обозначается как $\square$) и один или несколько других символов.
2. **Голова:**
 - Перемещается вдоль ленты.
 - Может считывать символ с текущей ячейки.
 - Может записывать символ в текущую ячейку.
 - Может двигаться влево или вправо по ленте.
3. **Управляющее устройство:**
 - Находится в одном из конечного множества состояний.
 - Содержит таблицу переходов, определяющую действия на основе текущего состояния и символа под головой.

Описание работы машины Тьюринга

Машина Тьюринга определяется следующим образом:

- Q – конечное множество состояний.
- Σ – конечный алфавит, содержащий символы, которые могут быть записаны на ленте.
- q_0 – начальное состояние ($q_0 \in Q$).
- q_F – конечное (или принимающее) состояние ($q_F \in Q$), если машина останавливается.
- δ – функция переходов: $\delta: Q \times \Sigma \rightarrow Q \times \Sigma \times \{L, R, N\}$ где L – движение головы влево, R – движение вправо, N – отсутствие движения.

На каждом шаге:

1. Машина находится в состоянии $q \in Q$
2. Голова считывает символ $\sigma \in \Sigma$ с текущей ячейки.
3. На основе $\delta(q, \sigma)$, машина:
 - Переходит в новое состояние $q' \in Q$
 - Записывает символ $\sigma' \in \Sigma$ в текущую ячейку.
 - Перемещает голову влево, вправо или остаётся на месте.

Пример работы машины Тьюринга

Рассмотрим простую машину Тьюринга, которая инвертирует биты на ленте (заменяет 0 на 1 и 1 на 0), и останавливается при встрече пустого символа $\square$

Алфавит: $\Sigma = \{0, 1, \square\}$

Состояния: $Q = \{q_0, q_1, q_F\}$

Начальное состояние: q_0

Конечное состояние: q_F

Функция переходов δ $\delta(q_0, 0) = (q_0, 1, R)$ $\delta(q_0, 1) = (q_0, 0, R)$ $\delta(q_0, \square) = (q_F, \square, N)$

Описание:

- Если машина в состоянии q_0 и читает 0, она записывает 1 и движется вправо.
- Если машина в состоянии q_0 и читает 1, она записывает 0 и движется вправо.
- Если машина в состоянии q_0 и читает $\square$, она переходит в состояние q_F и останавливается.